

Seizing the means of machine listening/ learning and agentic AI: creative practices with conTorchionist and vibe composing

Tomando os meios de escuta/aprendizado de máquina
e IA agêntica: práticas criativas com conTorchionist e
vibe composing



José Henrique Padovani

Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais, Brasil
jhp@ufmg.br



Vinícius Cesar de Oliveira

Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais, Brasil
oviniuscscar@gmail.com

Abstract: This article addresses the challenges posed by the hegemony of commercial, “black-boxed” Artificial Intelligence (AI) tools in musical creation, aiming to counteract the alienation inherent in Software as a Service (SaaS) platforms. Drawing on the concepts of mechanology and technodiversity, we advocate a critical and inventive engagement with technology through the appropriation of machine-listening and learning infrastructures. To this end, we present conTorchionist, an open-source, nomadic, and cross-platform library built upon libtorch, designed to bridge the gap between non-real-time research (Python) and real-time performance environments (Pure Data, Max, SuperCollider). Furthermore, we introduce the concept of ‘vibe composing’ as a methodology within research-development-creation and practice-based research in music, where the coding of algorithms assisted by agentic AI constitutes a fundamental poetic gesture. This approach is exemplified through two musical works, *second nature* (2025) and *Eutrópia III* (2025), which demonstrate how the development of idiosyncratic technical objects can engender

unique aesthetic outcomes, ranging from spectral spatialization to neural control of latent spaces. Finally, we reflect on how seizing these technical means allows artists, particularly in the context of the Global South, to assert epistemic and technical sovereignty, fostering an ecology of invention that resists the standardization of the algorithmic age.

Keywords: Machine Listening. Interactive Music Systems. Research-creation. Technodiversity. Agentic AI.

Resumo: Este artigo aborda os desafios impostos pela hegemonia de ferramentas comerciais de Inteligência Artificial (IA) do tipo “caixa-preta” na criação musical, visando contrapor a alienação inerente às plataformas de Software como Serviço (SaaS). A partir dos conceitos de mecanologia e tecnodiversidade, defendemos um engajamento crítico e inventivo com a tecnologia, por meio da apropriação de infraestruturas de escuta e de aprendizado de máquina. Para tanto, apresentamos a conTorchionist, uma biblioteca de código aberto, nômade e multiplataforma baseada em libtorch, projetada para unir a pesquisa em tempo diferido (Python) e ambientes de performance em tempo real (Pure Data, Max, SuperCollider). Além disso, introduzimos o conceito de “vibe composing” como uma metodologia no âmbito da pesquisa-desenvolvimento-criação e da pesquisa baseada na prática musical, na qual a programação de algoritmos assistida por IA agêntica constitui um gesto poético fundamental. Essa abordagem é exemplificada por meio de duas obras musicais, *second nature* (2025) e *Eutrópia III* (2025), que demonstram como o desenvolvimento de objetos técnicos idiossincráticos pode engendrar resultados estéticos únicos, variando da espacialização espectral ao controle neural de espaços latentes. Por fim, refletimos sobre como a tomada desses meios técnicos permite a artistas, particularmente no contexto do Sul Global, afirmar a

sua soberania epistêmica e técnica, fomentando uma ecologia de invenção que resiste à padronização da era algorítmica.

Palavras-chave: Escuta de Máquina. Sistemas Musicais Interativos. Pesquisa-criação. Tecnodiversidade. IA Agêntica.

Submitted on: February 18, 2026

Accepted on: March 5, 2026

Published on: April 2026

1 Introduction

In contemporary discourse, Artificial Intelligence (AI) is frequently personified as a mythical, omnipresent entity that, despite being created through technical and scientific processes, is often described as an autonomous and self-evolving force that operates beyond human control: a framing that is described by Campolo and Crawford (2020) as “enchanted determinism”. This narrative, however, obscures the material reality of AI as a technopolitical mechanism embedded within what some scholars term as *platform capitalism* (Srnicek, 2017) or *technofeudalism* (Varoufakis, 2023). In this context, AI is not an autonomous force but a set of technical objects and infrastructures that are designed, developed, and deployed by specific actors with particular interests. Far from being open and accessible, prevalent commercial AI tools are predominantly distributed as opaque *Software as a Service* (SaaS) platforms that enforce new forms of digital colonialism and labor precarization (Regilme, 2024).

In the musical domain, this hegemony manifests through “black-boxed” tools that reduce the creative process to the operation of closed systems. Employed as a “service”, the tools of artistic production are not merely externalized, but standardized and alienated from the artist’s very creative speculation, dialectical engagement, and technical experimentation. While these services certainly allow for some degree of customization, they ultimately limit the artist’s agency to predefined affordances and pragmatic constraints. The consequence of this is the confinement of artistic creativity within the boundaries of a consumer relationship, with commercial products that are designed to maximize data extraction and profit rather than to foster creative exploration and technical sovereignty. By preventing artists from accessing, modifying,

or understanding the underlying technical processes, these services foster a “proletarianization” of knowledge (Stiegler, 2010) and a psychophysical alienation from the technical object (Simondon, 1989).

To counter this tendency, we advocate for a critical and inventive engagement with AI tools that resists the standardization of commercial offerings and promotes the epistemic and technical sovereignty of artist-researchers. In this article, we present some outcomes of our research that synthesizes critical inquiry, technical experimentation, and artistic creation to foster such engagement within music composition and *research-development-creation*. By generating artistic, critical, and technical outcomes, this approach resists industrial standards and advocates for *technodiversity* (Hui, 2020, 2021), thereby promoting the emergence of diverse, contextual, and relational modes of knowing and making. From this perspective, tool-building is not an ancillary task but an integral component of the creative process, enabling the artist-researcher to maintain a critical dialogue with technical objects. This practice defies the limitations of commercial tools and fosters the epistemic and technical autonomy of the creative practitioner.

The article is structured as follows: in the second section, we reframe the history of *machine listening* research from symbolic interaction to audio analysis through a critical lens. We adopt the concept of “listening *with* machines”, proposed by Emmerson (2015), but shift its focus from the archetype of the “intelligent assistant” to a more dialectical and inventive relationship with technical objects. Instead of positioning the machine as a passive tool or a mimetic entity that serves the artist’s intentions, we affirm the potential of developing, adapting, and experimenting with technical objects as a form of creative and inventive engagement that fosters a prosthetic

perspective. In this view, technical objects are crafted to ensure the artist's epistemic sovereignty, resonating with Gilbert Simondon's notion of *mechanology* (Simondon, 1989) and Yuk Hui's call for *technodiversity* (Hui, 2020).

In the third section, we present *conTorchionist*, an open-source, nomadic, and flexible toolset for machine listening and learning. Unlike closed commercial tools, *conTorchionist* is designed for experimentation and usage across multiple platforms, including *Pure Data*, *Max*, *SuperCollider*, and *Python*. It supports diverse usage contexts such as real-time and non-real-time processing, allowing artists to maintain a critical and inventive relationship with the technical objects that mediate their creative process.

In the fourth section, we discuss the artistic side of this approach, in which the employment of *conTorchionist* and the development of technical artifacts using agentic AI take on a central role in the creative process. We present two recent works by the authors, *second nature* (2025) and *Eutrópia III* (2025), which explore the relationship between listener and machine, the role of spatiality as a material force, and the potential of practices of "vibe composing". Within this methodology, coding computer algorithms to create real-time interactive music tools is not a technicality or a secondary activity, but a fundamental creative gesture and a critical engagement with the appropriation of AI resources.

In conclusion, we reflect on the artistic, methodological, and political implications of this approach, advocating for a critical and inventive engagement with AI instead of positions of either uncritical acceptance or misoenism. We also address the practical challenges of research-creation regarding the development of such tools within the Global South context. Finally, we point towards future work, focusing on the continued development of the tool in research-creation and practice-based research scenarios, as well as the enhancement of documentation and distribution strategies for the *conTorchionist* wrappers

2 Seizing the means of machine learning/listening

The anthropomorphization of “listening” into the realm of machines anticipates, in the context of experimental music, computer music, and artistic research using real-time audio processing, the questions that are now being raised in distinct contexts regarding the personification of AI-related technologies. The historical trajectory of what is commonly referred to as “machine listening” evolved from the computational manipulation of symbolic representations of music (often MIDI) to the sub-symbolic domain of digital signal processing (DSP). This transition, described by Rowe (2009) as a shift from the *first wave* of symbolic interactive systems to a *second wave* of audio-based analysis, fundamentally altered how machines are configured to process musical information. In the context of *Interactive Music Systems* (IMS), this paradigm relies on the duality of “listening” and “acting” (Rowe, 1992).

Within this framework, the machine is designed to simulate a capacity to “listen” to the *input* from a performer and engage in a musical interaction, generating *outputs* that are often perceived as exhibiting a certain level of “intentionality” (Lewis, 1999, 2000). It is worth noting that, since the early days of IMS, this interaction was already envisioned as a “subject-subject” negotiation, exhibiting traces of psychological projections into the anthropomorphized machine that were already identified by Sherry Turkle in 1984 while studying the relationship between humans and the hardware and software of the time (Turkle, 2005).

As *machine listening* consolidated as a topic of research, it was codified through analytical lenses often modeled on biological mechanics. In Computational Auditory Scene Analysis (CASA), algorithms were developed to perform a reverse engineering of the human auditory system, emulating the segregation principles described by Bregman (1990) with the explicit goal of matching human performance standards (Wang; Brown, 2006). Parallel to this, Music Information Retrieval (MIR) focused on

extracting high-level features to organize large-scale musical data (Downie, 2003). However, while this technical tradition tends toward an anthropocentric bridging of raw audio and symbolic processing, *machine listening* simultaneously unfolded as a critical aesthetic practice. Rather than merely mimicking human audition, works in this domain explicitly problematize the human-machine relationship, proposing alternative, non-mimetic roles for the machine in creative processes (Caesar, 2013; Collins, 2006, 2007, 2011; Emmerson, 2007; Jehan, 2005).

Building upon these critical lineages, a shift from a *mimetic* to a *prosthetic* perspective repositions *machine listening* as a site of critical and inventive mediation. While Emmerson (2007) introduced the concept of “listening *with* machines” to describe a collaborative relationship between human and machine, this notion can be further expanded to move beyond the archetype of an “intelligent assistant”. While this notion certainly had other implications in the context of the early 2000s, it is now often co-opted by the marketing of commercial AI tools that promise to be “intelligent” collaborators, assistants, or co-creators. This trope is nowadays not only becoming exhausted in current AI marketing but is also increasingly met with skepticism due to inherent issues of privacy and data extraction performed by the so-called “intelligent” tools of tech giants, reproducing what Gilbert Simondon identified as a “scheme of servitude” (Simondon, 1989, p. 127).

The notion of “listening *with* machines” is thus articulated in our work as a critical and inventive engagement with technical objects crafted as prosthetic extensions. In this framework, these objects are developed by artists-developer-researchers specifically to mediate, modulate, and experiment with their relationship to sonic materials and the technical milieu itself. This perspective emphasizes the active role of the practitioner in shaping the technical objects that facilitate this mediation and modulation. Rather than pursuing the reverse engineering of human cognition through a passive tool or an “intelligent”

servant, these objects are actively crafted within practice-based research as prosthetic extensions and transductive channels. Such extensions do more than merely expand the perceptual and agential fields; they fundamentally transform and reconfigure them, enabling new modes of engaging with the sonic environment and the technical objects that mediate the creative process.

This perspective is directly informed by Simondon's observation that "alienation" is not restricted to the property relations of the means of production (Marx, 1973, p. 388), but also extends to the modes of engagement with technical objects, to their inclusion in the realm of cultural artifacts, and to the epistemic and psycho-physiological relationship between humans and machines (Simondon, 1989, p. 248). While Simondon posits that overcoming alienation requires a technical culture where humans understand and participate in the genesis of the technical objects, the contemporary technological landscape presents new challenges to this project. As Hui (2019, 2020) observes, the current global context is dominated by the notion of a *monotechnology*: the belief in a single, universal line of technological development. This perspective is implicitly reinforced by the discourses and ideologies surrounding AI and the race for technological supremacy. A more diverse and pluralistic approach to technology, as advocated by Hui's concept of *technodiversity*, parallel to the call for the diversity of life forms (*biodiversity*) and the diversity of knowledge systems (*noodiversity*), is essential to counteract these homogenizing narratives and, in the context of artistic practices, to reclaim the technical objects as sites of invention and creative agency (Hui, 2024).

Seizing the means of machine learning and listening is framed here as a resistance to this homogenization and a return to the technical object as a primary site of invention within a *research-development-creation* (R-D-C) paradigm.¹ If, as Lachenmann (1996) contends, to “compose is to build an instrument,” then creative practices involving *machine listening* and *machine learning* inherently imply the critical and compositional design of prosthetic means of technical mediation that comprise an expanded notion of “instrument” in the context of interactive music.

Through the development of *conTorchionist* and the practice of “vibe composing,” the very technical objects and methods that mediate the creative process and musical interaction are constructed. Within this recursive framework, the development of specialized tools using agentic AI or the diverse modules that comprise *conTorchionist* are not mere technical tasks but fundamental creative gestures that establish new modes of listening and interaction. These methods, applied here to the rapid prototyping of idiosyncratic computer music tools with AI resources in a context analogous to *vibe coding* (Ray, 2025), function as foundational creative gestures that integrate code, sound, and theory. By cultivating a material understanding of the technical milieu, these practices allow the artist to bypass the standardization of commercial tools and to counteract the epistemic opacity often associated with the “AI industry.” Furthermore, this approach extends beyond the isolated “artwork” or individual authorship, as the technical objects developed are shared as modular, open-source resources to invite further exploration across diverse contexts and languages.

¹ The R-D-C framework expands upon *practice-based research* (Candy; Edmonds; Vear, 2022) and *research-creation* (Gosselin; Le Coguiec, 2006) by positioning technical development not as a preparatory or ancillary stage, but as a central epistemic activity related to both the research questions and the artistic practice. It assumes that theoretical investigation and artistic invention are inextricably mediated by the production of new technical artifacts, where the development of software and hardware is tightly interwoven with the creative process and with critical reflection on the questions raised by the artistic practice.

3 conTorchionist: A flexible nomadic library for exploring machine listening/learning in multiple platforms, languages, and time-contexts²

Diverse tools, libraries, and frameworks have been developed to support the creative, investigative, and commercial use of *machine listening* techniques in various contexts, such as music production, recommendation systems, and interactive applications. Many of these tools rely on neural networks and other artificial intelligence resources, dramatically expanding what may be done with them.

Despite the proliferation of *machine listening* tools, a significant gap remains regarding cross-platform compatibility and hardware-agnostic flexibility. Few tools are available for use across multiple platforms, GPU/CPU architectures, and programming languages. This limitation hinders the application of the same algorithms and techniques in diverse contexts, such as real-time audio processing in interactive music systems like *Pure Data* (PD), *Max*, and *SuperCollider* (SC), or non-real-time audio analysis in *Python*. Consequently, it restricts workflows and exploratory practices that could benefit from processing large volumes of sound files for model training³. Furthermore, although some of these tools provide neural processing capabilities across multiple operating systems, a notable disparity exists: while audio processing units (UGens, externals, classes, etc.) are diverse and abundant, enabling highly flexible audio processing chains, the same is not true for neural network and machine learning algorithms, especially within computer music environments. Typically, the artist has very limited control over or access to the logic of model building or its internal mechanisms. This can restrict creative

² This section is an expanded and revised version of the original paper *conTorchionist: A flexible nomadic library for exploring machine listening/learning in multiple platforms, languages, and time-contexts* (Padovani; Oliveira, 2025), which was published in the proceedings of the 19th Brazilian Symposium on Computer Music, 2025.

³ A remarkable and inspiring exception is *FluCoMa*, which allows for cross-platform use in *Pure Data*, *Max*, *SuperCollider*, and CLI, with community-built wrappers also available for *Python* and an extension for the *REAPER* DAW, *ReaCoMa*.

exploration and experimentation, leading to more alienated relationships between the artist and the technical tool that fall into the category of a *black box*.

Driven by the authors' artistic-investigative need to integrate workflows across disparate domains, particularly bridging *Python* and *PD*, *conTorchionist* emerges as an open-source library and an ongoing research project exploring nomadic platforms for machine listening and learning. Built upon *libtorch*, the C++ distribution of *PyTorch*, the library integrates audio analysis, machine learning, and tensor-based computation into a unified and portable toolkit. By utilizing a single shared core across diverse interfaces, including *Pure Data*, *Max*, *SuperCollider*, and *Python*, the library provides a versatile resource for interactive music systems, computer music research, and sonology. This architecture restores the technical object's plasticity while enabling efficient computation across various hardware architectures, such as NVIDIA/CUDA and Apple/MPS, allowing for a flexible and exploratory engagement with machine listening workflows in environments traditionally focused on signal processing.

3.1 Related work

A number of objects/UGens, libraries, packages, and extensions have been developed to support *machine listening* for audio analysis, processing, and synthesis in various contexts. Some of these tools are specifically designed for music creation environments, such as *Pure Data*, *Max*, and *SuperCollider*, while others are more general-purpose libraries intended for use in *Python* or other programming languages. We will briefly outline the native and the most popular extensions available in these environments to contextualize some choices regarding the *conTorchionist* design⁴.

⁴ This section references the following major versions: *Pure Data* 0.55, *Max* 9, *SuperCollider* 3.13, and *Python* 3.13.

In *Pure Data* (and related environments, like *plugdata* and *Purrrdata*), the landscape of native audio analysis is dominated by the foundational objects developed by Miller Puckette, which, although often distributed as extensions, constitute the de facto standard in the *vanilla* distribution. These include [env~] for dB RMS analysis, [bonk~] for spectral onset detection, and the versatile [fiddle~] and [sigmund~] for pitch tracking and partial analysis (Puckette, 1997). *Max* incorporates similar functionalities through native objects like [retune~] for pitch shifting, [loudness~] for LUFS-based analysis, and tools such as [peakamp~] and [levelmeter~]. Both environments allow for the construction of complex analysis chains by leveraging FFT/RFFT processes and buffer-based operations, supported by visualization tools like [spectroscope~] and [waveform~].

SuperCollider provides a more extensive set of native Unit Generators (UGens) dedicated to audio analysis and spectral processing. The environment features a dedicated “Analysis” suite comprising over seventeen self-descriptive UGens, ranging from amplitude followers like AmpComp and Peak to more specialized MIR tools such as BeatTrack, Keytrack, MFCC, and Onsets. Additionally, its FFT-related library includes common feature extraction processes such as SpecCentroid and SpecFlatness, making it a robust platform for real-time descriptor analysis without requiring external dependencies. In contrast, *Python*, being a general-purpose language, does not ship with native audio-specific modules, relying entirely on its ecosystem of third-party packages for signal processing (Supercollider, 2025).

The functional range of these environments is vastly expanded by community-developed extensions. In the realm of visual programming, *timbreID* for *Pure Data* provides a comprehensive suite for extracting spectral and temporal features, including the [timbreID] object for feature vector comparison and concatenative synthesis (Brent, 2009; Brown; Wang, 2015). Similarly, *Zsa.Descriptors* has become a standard

for *Max* users, offering a wide array of descriptors that cover energy, dynamics, and statistical spectral measures (Malt; Jourdan, 2009). In the *SuperCollider* ecosystem, Nick Collins' *SCMIR* library bridges the gap between synthesis and Musical Information Retrieval by offering tools for beat tracking and onset detection alongside additional executables for machine learning tasks such as Gaussian Mixture Models, Hidden Markov Models, and Neural Networks (Collins, 2011).

The *FluCoMa* (Fluid Corpus Manipulation) project, led by Pierre Alexandre Tremblay, represents a significant milestone in multi-environment toolsets, providing a unified C++ core for *Max*, *PD*, and *SuperCollider*, as well as a command-line interface (CLI). It facilitates complex workflows involving audio decomposition and machine learning, with a strong focus on community-driven research and high-performance algorithms. While *FluCoMa* lacks an official *Python* interface, community wrappers have extended its reach into *Python* and REAPER (Moore *et al.*, 2021; Tremblay *et al.*, 2019; Tremblay *et al.*, 2022; Tremblay; Roma; Green, 2021). Within the *Python* ecosystem itself, *Librosa* and *Essentia* stand as the primary references for MIR research. *Librosa* is widely celebrated for its ease of use in offline spectral and temporal analysis, whereas *Essentia* prioritizes scalability and real-time efficiency through its C++ implementation and *Python* bindings (Bogdanov *et al.*, 2013; McFee *et al.*, 201).

Modern developments have increasingly focused on the integration of deep learning. *PyTorch* and its audio-specific extension *torchaudio* provide a flexible framework for building end-to-end pipelines that leverage GPU acceleration and tensor computation (Hwang *et al.*, 2023; Paszke *et al.*, 2019; Yang *et al.*, 2022). To bridge the latency gap between *Python*-trained models and real-time performance, the *nn~* external for *Max/PD* (and its *SuperCollider* port *nn.ar*) utilizes the *libtorch* C++ backend. This allows for the direct inference of RAVE (Realtime Audio Variational

AutoEncoder) models within creative environments, providing a high-performance interface for neural audio synthesis and complex signal mapping (Caillon; Esling, 2021, 2022).

3.2 Overview of *conTorchionist*

Despite these advances, a unified solution for cross-platform, architecture-agnostic deployment of complex *PyTorch* models remains an open challenge, a gap *conTorchionist* aims to fill. While acknowledging the foundational role of tools like *FluCoMa*, which serves as a key inspiration for our approach, *conTorchionist* occupies a distinct niche focused on nomadism, flexibility, interoperability, and integration of machine listening/learning workflows across diverse languages/environments, operational systems, architectures, and temporal contexts. The library takes advantage of the inherent portability of C++ and the comprehensive capabilities of *libtorch/PyTorch*, a mature and industry-standard framework for machine learning that abstracts underlying hardware complexities to grant access to advanced acceleration capabilities across Windows, macOS, and Linux. This ensures that the same DSP and neural logic can run efficiently on both Intel and ARM architectures, utilizing CPU or GPU resources (including NVIDIA CUDA and Apple Metal Performance Shaders, MPS) without necessitating environment-specific re-implementations.

Beyond hardware flexibility, the library aims to dissolve the boundaries between temporal domains and software ecosystems. The idea is to enable a seamless workflow where models developed for deferred-time analysis in *Python* can be employed for real-time interaction in *Pure Data*, *Max*, or *SuperCollider*. This algorithmic consistency fosters an exploratory approach to machine listening, providing a modular suite of objects that can be assembled into complex signal chains in artistic or research contexts. By unifying the technical object across platforms, *conTorchionist* not only enhances accessibility but also encourages a deeper engagement with the underlying

processes of machine listening and learning, providing artists and researchers with the tools to build their own technical mediations and to explore the creative potential of these technologies.

To support this versatility, the system architecture is bifurcated into a platform-agnostic core library and environment-specific wrappers. The core serves as a unified C++ interface to *libtorch*, encapsulating the framework's functionality into two main components: processors, which handle the core logic of audio analysis and machine learning, and utils, which manage device selection and tensor manipulation. Surrounding this core, the wrappers provide the necessary bindings to translate these functionalities into the idiomatic structures of each target language, handling tasks such as argument parsing and type conversion. In the current development stage, efforts are focused on validating the core processors through internal tests and the *Pure Data* implementation.

3.3 Processors and objects

Currently, the *conTorchionist* library provides a suite of spectral analysis tools, neural network utilities, and some more idiosyncratic processors and wrappers developed during the creative process of the pieces and artistic experiments presented in Section 4. The development effort focuses mainly on ensuring these core processors offer high customizability and flexibility, allowing their results to be comparable with established libraries such as *librosa* and *FluCoMa*. While the initial set of processors covers fundamental analysis tasks, the architecture is designed to be extensible and modular.

The core-wrapper structure allows for the addition of new processors and wrappers in a scalable manner, allowing for using agentic AI to rapidly prototype and implement new functionalities as they arise from artistic needs. This design philosophy has enabled the development of custom processors

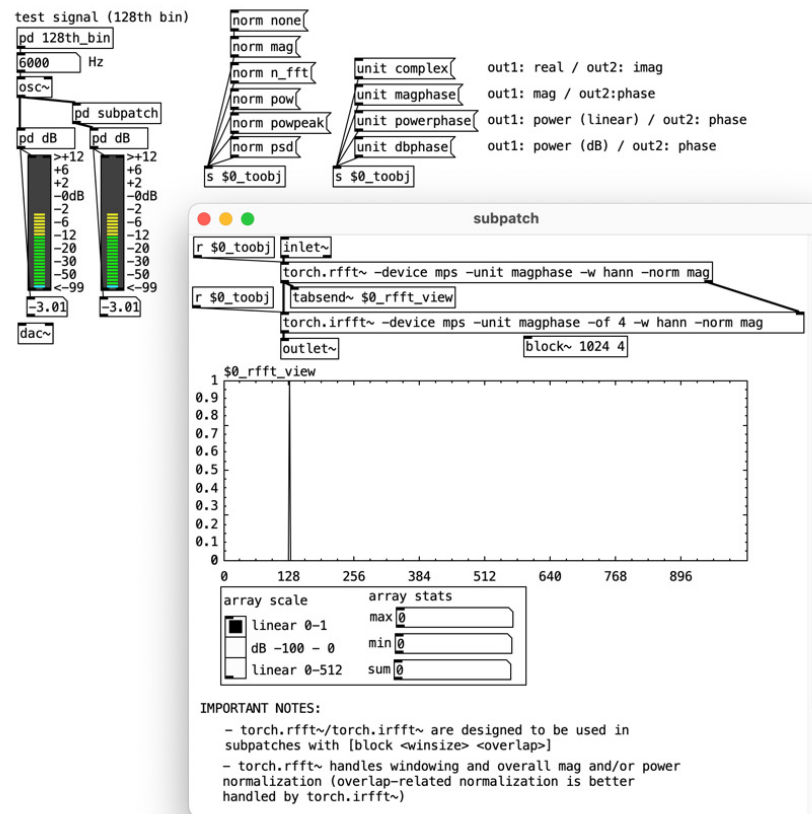
that address quite specific challenges in spectral tracking and spatialization, which were required for the piece *second nature* (2025) and which are not found in existing libraries. At the same time, the modularity and the unified core around *libtorch* allow for the integration of neural network models and algorithms, allowing for workflows that combine traditional patch-based signal processing with *Python*-based model development and training.

3.3.1 Audio and Machine Listening Processors

The core of the library features a modular set of signal processors implemented in C++ and exposed to environments like *Pure Data* and *Max* through dedicated wrapper objects. These processors are device-agnostic, supporting seamless execution on CPUs and GPUs (via CUDA or Metal Performance Shaders). Currently, the wrappers are mostly developed for *Pure Data*, with *Max*, *SuperCollider* and *Python* implementations in progress.

The *RFFTProcessor* and *IRFFTProcessor* classes utilize optimized algorithms from *libtorch* to perform real and inverse Fast Fourier Transforms, supporting various windowing, overlap and normalization configurations (Figure 1). Building upon these, the *SpectrogramProcessor* employs an auxiliary *CircularBuffer* to manage audio streams independently of the host environment's block size, ensuring glitch-free real-time spectrogram computation. For timbre feature extraction, the *MelSpectrogramProcessor* and *MFCCProcessor* handle frequency mapping to the mel scale and cepstral coefficient extraction, respectively, using dynamically constructed filterbanks.

Figure 1 - The [torch.rfft~] and [torch.irfft~] objects in PD



Source: Elaborated by the authors

Description of the image: Pure Data patch and subpatch showing the [torch.rfft~] and [torch.irfft~] performing for real and inverse RFFT processing, with hanning windowing and 4 overlaps in magnitude+phase mode, using Apple MPS GPU acceleration.

Expanding this set to meet specific artistic needs, such as those required by the work *second nature*, gesture and spatial analysis processors were introduced. The Spectral Trails Processor implements peak detection with slope-based tracking and configurable attack/decay envelopes, allowing for the isolation of persistent spectral peaks which trigger slowly decaying trails in the output. This strategy was strongly influenced by the adaptation of the spectral analysis and peak/f0/partial tracking techniques employed by Miller Puckette in the [sigmund~] *Pure Data* object (Puckette, 2005).

Its spatial variation, *AmbisonicSpectralTrails*, applies this logic to 3D soundfields in *Ambisonics* format. Also developed with a great help of agentic AI to translate, interpret, and rewrite algorithms, this processor was developed by adapting and combining the same logic used in *SpectralTrailsProcessor* with *Ambisonic* encoding techniques derived from the IEM Plugin Suite by Frank; Zotter; Sontacchi (2015), allowing for triggering spatialized spectral trails based on the frequency peaks of the soundfield.

Additionally, the *MicArraySpecAnalyzer* offers spatial analysis for microphone arrays, enabling the computation of directivity patterns, while the *CorpusMatcher* facilitates real-time similarity matching, bridging feature extraction with concatenative synthesis workflows.

Table 1 – conTorchionist main Processors and Pure Data wrappers (objects).

C++ Class (Core)	Pure Data Object	Functional Description
Audio / DSP Processors and Objects		
RFFTProcessor	[torch.rfft~] / [torch.irfft~]	Real FFT (direct and inverse) with configurable windowing, overlapping and output modes
SpectrogramProcessor	[torch.spectrogram~]	Spectrogram (magnitude, power, dB scales) with frame buffering.
MelSpectrogramProcessor	[torch.melspectrogram~]	Mel-scale spectrogram using triangular filter banks.
MFCCProcessor	[torch.mfcc~]	Mel-Frequency Cepstral Coefficients (DCT compressed features).
RMSOverlapProcessor	[torch.rms_overlap~]	Sliding RMS energy analysis with customizable window overlap.
SpectralTrailsProcessor	[torch.spectrals~]	Spectral peak processor triggered by dB thresholds

AmbisonicSpectralTrails	[torch.amb.spectrals~]	Ambisonic spectral peak processor triggered by dB thresholds
MicArraySpecAnalyzer	[torch.mic_array_spectral_analyzer~]	Spatial direction estimation and strength analysis from mic arrays.
CorpusMatcher	[torch.corpus.matcher~]	Feature vector similarity search in pre-computed corpora.
ArrayPlayer	[torch.arr.player~]	Play multichannel files and arrays.
<i>Machine Learning Processors and Objects</i>		
TorchTSProcessor / TorchTSProcessorSignal	[torch.ts] / [torch.ts~]	Loading and real-time inference of TorchScript models in control and audio rate.
SequentialProcessor	[torch.sequential]	Sequential container for stacking neural layers.
LinearProcessor / LinearAProcessorSignal	[torch.linear] / [torch.linear~]	Dense (fully connected) neural network layers.
MHAProcessor	[torch.mha]	Multi-Head Attention mechanism for sequence processing.
ActivationProcessor	[torch.activation]	Nonlinearities (ReLU, Sigmoid, Tanh, etc.).
Ls2TensorProcessor	[torch.ls2tensor]	Converts lists/audio streams into tensors for network input.
ReshapeProcessor	[torch.reshape]	Tensor reshaping and dimension manipulation.

Source: Elaborated by the authors

3.3.2 Neural Network Processors

The machine learning-related objects are organized into two main workflows: (1) real-time inference of complex pre-trained models, and (2) interactive building and training of neural networks directly within the musical environment.

The TorchTSProcessor and its audio signal counterpart, TorchTSProcessorSignal (exposed as [torch.ts] and [torch.ts~]), allow for loading and executing *TorchScript* models.

A fundamental feature of the audio object [torch.ts~] is its asynchronous processing mode. By using the -async flag, the object instantiates an internal circular buffer that accumulates signal blocks and performs model inference in a separate thread. This design allows for the execution of computationally intensive models without blocking the main audio thread, thereby preventing audio dropouts.

For the prototyping and construction workflow, the SequentialProcessor adapts PyTorch's torch.nn.Sequential class. It acts as a named container where layers processed by other objects (such as [torch.linear], [torch.mha], [torch.activation], and [torch.reshape]) can be dynamically added. To feed these networks with audio data, the Ls2TensorProcessor (exposed as [torch.ls2tensor]) accumulates feature streams, such as MFCC output, creating formatted tensors that can be accessed directly by the sequential container for training or inference tasks.

4 Creative Projects using *conTorchionist* and agentic AI

The *conTorchionist* library was developed in close alignment with its practical application in creative contexts, specifically within *research-development-creation* (R-D-C) and *practice-based research* workflows. Consequently, while technical and theoretical frameworks informed the library's overarching architecture, many specific design and implementation choices were directly shaped by the poetic and pragmatic requirements encountered during the compositional process and the exploration of creative and investigative possibilities.

Central to this approach is the integration of agentic AI as a catalyst for technical sovereignty. By utilizing Large Language Models (LLMs) through a methodology termed here as *vibe composing*, it became possible to rapidly prototype and

refine DSP, auxiliary, and machine learning processes, while simultaneously structuring a comprehensive architecture for the library and its compilation workflow. The term *vibe composing* refers to a creative practice in which algorithmic development is assisted by agentic AI. Here, the act of coding is not merely a technical ancillary but a fundamental creative gesture and a critical engagement with the appropriation of AI resources. Within this framework, coding is not entirely delegated to LLMs but becomes a dialogic process of refining, adapting, and experimenting with generated code, using existing software and libraries as references. In our creative practices, this methodology was instrumental in facilitating the analysis of external tools, the rapid prototyping of new algorithms, and the development of a flexible, cross-platform, open-source architecture for machine listening and learning.

This section delineates how these strategies were materialized through two distinct musical works in which we have not only employed the library, but created its technical artifacts as part of creative and investigative processes. In other words, more than simply exemplifying how *conTorchionist* was applied in creative contexts, these works illustrate how the development of the library and its components was deeply intertwined with our own artistic practices, consciously aimed at appropriating AI resources in a personal and inventive way.

First, we examine *second nature*, a work that explores situated technodiversity through the development of custom machine listening processors, a set of graphical user interface objects, and hybrid human-machine interactions. Second, we discuss *Eutrópia III*, which utilizes the library to navigate latent spaces and manage complex spatial trajectories. Both pieces exemplify not only the technical capabilities of the library and

of working with agentic AI, but also the potential of research-development-creation methodologies that foster a critical and inventive engagement with technology in a *technodiverse* and *prosthetic* perspective.

4.1 DSP processes and vibe composing in *second nature*

The collaborative project *second nature* (2025), for choir, *róí-róis* (traditional Brazilian friction instruments), and live electronics, was composed by José Henrique Padovani in a collaborative project with Miriam Akkermann, Michael Käßler, Mia Hadzikadunic, and the Singakademie Dresden. Developed in 2025 within the framework of the international collaboration research project *Listening, studying, and composing music with listening, learning, and composing machines: a technical, creative, and musicological*, the work explores themes of sound ecology, problematizing the dualities between nature/culture, culture/technology, and technology/nature. The piece examines the role of technical mediation in collective listening and sound production, alongside the potential for machine listening and interactive systems to cultivate novel forms of sonic interaction and ecological awareness.

As a research-development-creation project centered on auditory scores and interactive processes, *second nature* necessitated a suite of custom tools developed through a dialogic workflow involving agentic AI resources in practices that we denote as *vibe composing*, understanding that the “active pairing” approach of using natural language prompts to support part of the coding process, as described by Ray (2025), is not only a technical strategy but also a compositional one. This approach facilitated the rapid prototyping, refining, and debugging of technical artifacts using existing software and libraries as references, while ensuring that technical development and compositional experimentation mutually

informed one another. By leveraging AI as an auxiliary resource, it was possible to maintain an agile, iterative cycle where new tools were tested immediately against the piece's specific material and interactive requirements.

Under this methodology, several key systems were developed beyond the core *conTorchionist* library: a *WebRTC*-based audio streaming framework, a dedicated set of graphical user interface (GUI) objects, modules for *Ambisonics* encoding and spatialization, and tools for real-time spectral analysis of microphone arrays. The necessity for these specific artifacts emerged directly from collaborative experiments with the choir, ensuring that the resulting technical infrastructure was deeply rooted in the creative exploration of the work's central interactive mechanisms.

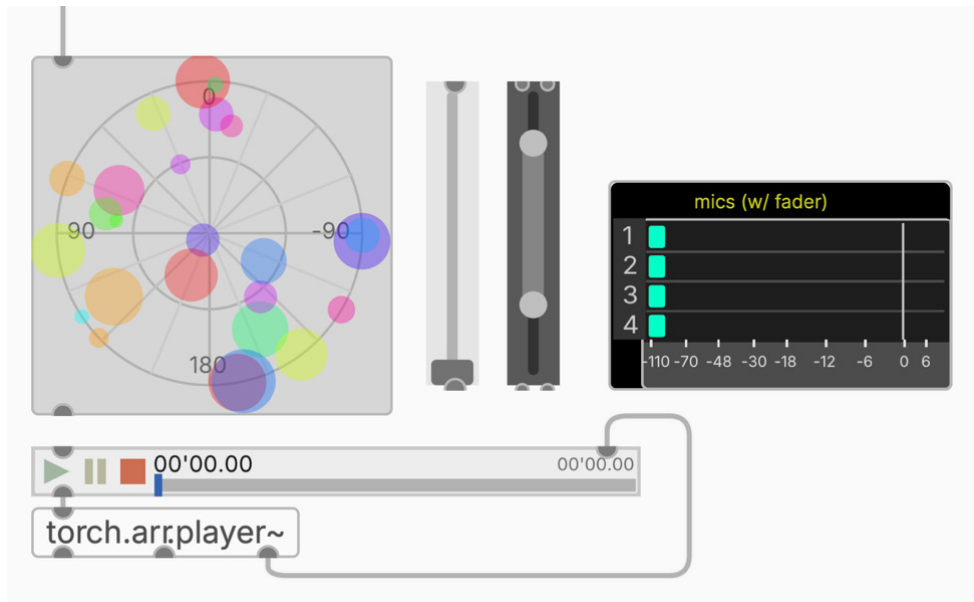
For instance, during experiments with the singers, two distinct systems were developed to facilitate real-time streaming of individualized aural cues and sound material. The first was a custom *WebRTC*-based⁵ multichannel streaming system that, combined with the Linux *PipeWire*⁶ sound server, distributed personalized audio cues to the singers' mobile devices via a locally hosted web application. The second system utilized the *Janus WebRTC Gateway* and a custom-built *Pure Data* object to send individualized audio channels to a *Dockerized*⁷ server instance. This latter system was designed to support both locally and remotely hosted streaming, opening possibilities for remote rehearsals and other forms of telematic musical interaction.

5 WebRTC (Web Real-Time Communication) is an open-source project providing web browsers and mobile applications with real-time communication via simple application programming interfaces (APIs), allowing audio and video communication to work inside web pages without the need for additional plugins.

6 PipeWire is a low-latency multimedia framework for Linux that handles audio and video routing and mixing, designed to replace both PulseAudio and JACK while providing improved security and performance for professional audio applications.

7 A "Dockerized" instance refers to software running within a Docker container, a lightweight, standalone, and executable package that includes everything needed to run an application (code, runtime, system tools, and libraries), ensuring consistency across different computing environments.

Figure 2 - Some of the GUI objects in the *ctgui* library, in *plugdata*



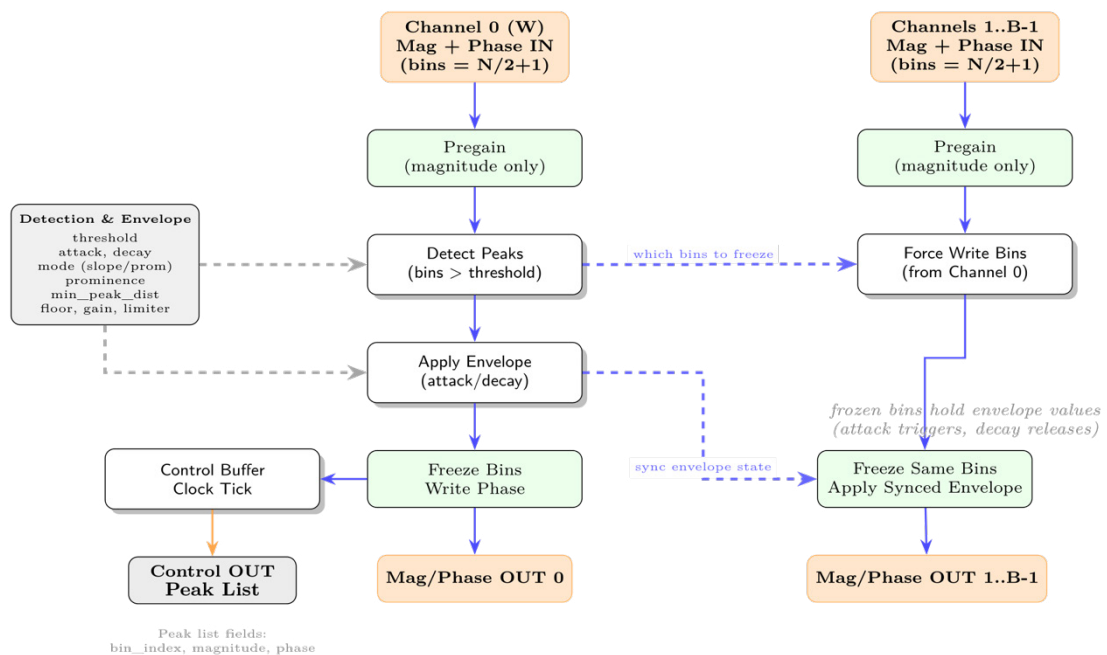
Source: Elaborated by the authors.

Top, from left to right: [ctgui.specradar], [ctgui.slider], [ctgui.rslider], and [ctgui.vu]; Bottom: ctgui.play controlling the multichannel array player [torch.arr.player~]

The development of the graphical user interface was further driven by the necessity of addressing the limitations inherent in *Pure Data* and *plugdata* native GUI objects. Notably, *Pure Data*'s GUI rendering is often aesthetically constrained and computationally inefficient; this presented a significant challenge given the work's requirement for real-time visual feedback of complex processes (VU meters, spatialization visualization, spatio-spectral representations, etc.) as well as control interfaces (sliders, range sliders, audio transport control, etc.). To circumvent these limitations, the *ctgui* library was developed to provide an extensible GUI ecosystem built entirely in *Lua* using the *pdlua* framework (Gräf; Schoen; Wesch, 2025). Conceived as a complement to *conTorchionist*, *ctgui* provides objects that are visually consistent and CPU-efficient. For example, all objects feature internal logic that decouples the visual rendering rate from the data processing rate. Furthermore, the sliders (*ctgui*.

slider and `ctgui.rslider`) include dedicated easing processes, enabling fluid and responsive interfaces without requiring additional objects such as `[line~]` or `[vline~]`. Beyond generic controls, the library includes specialized objects such as `ctgui.amb.directivity`, which provides a polar representation of directivity patterns in the Ambisonic field, and `ctgui.speccrader`, a radar-like display for visualizing spectral content retrieved by *conTorchionist's* `[torch.mic_array_spectral_analyzer~]` object (Figure 2)⁸.

Figure 3 - The `[torch.amb.spectrals~]` DSP logic



Source: Elaborated by the authors.

The processor freezes spectral peaks in time and space when a frequency peak is detected above a user-defined threshold, generating a slowly decaying "spectral trail" that retains the original spatial position of the sound source.

Regarding spatialization, as the piece explores space in both auditory and physical dimensions, with performers moving freely through the audience, the need for flexible tools drove the

⁸ The *ctgui* library is available at <https://github.com/ecrisufmg/ctgui>. Accessed on February 14, 2026.

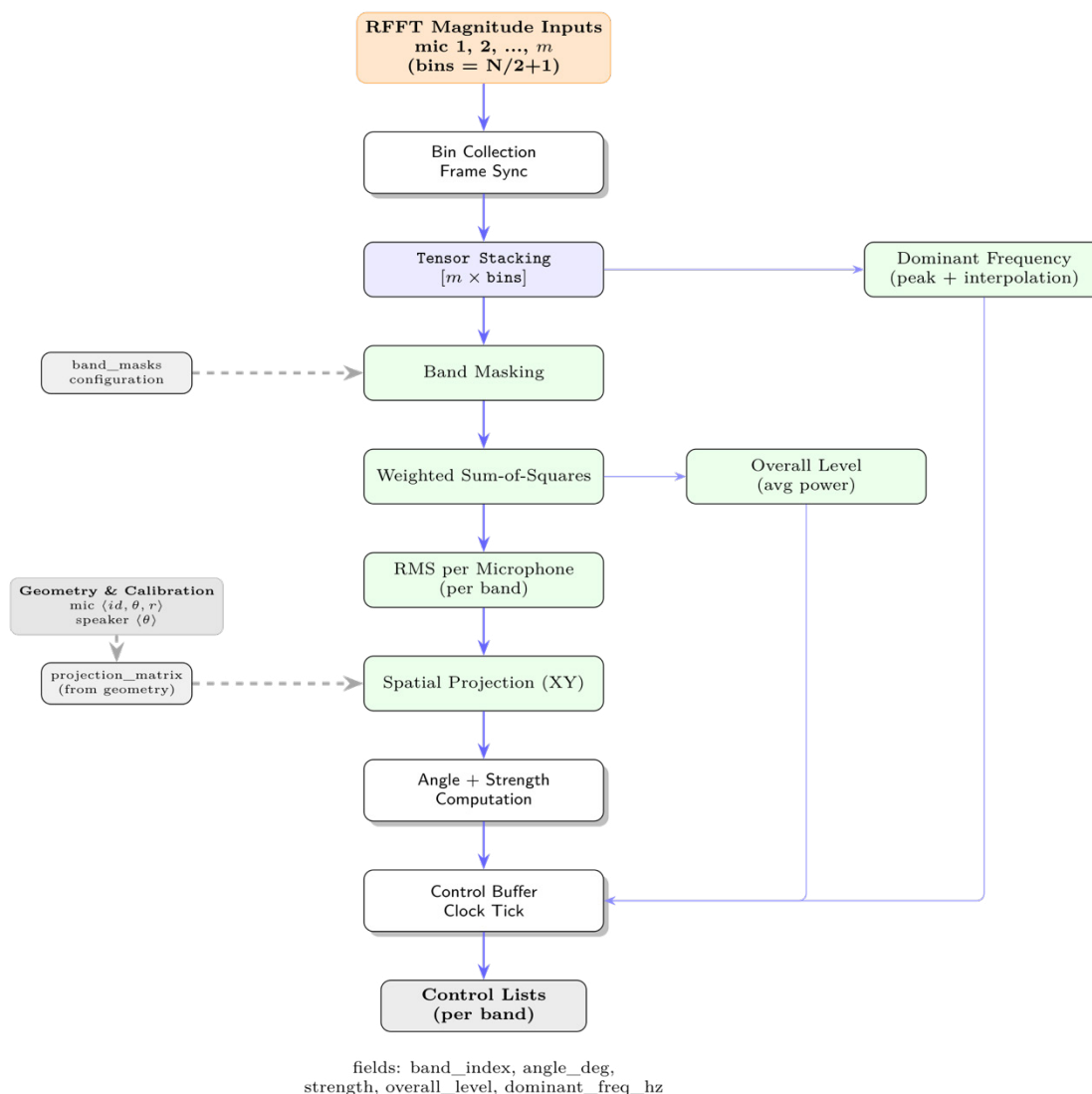
development of custom Ambisonics encoding and spatialization objects. These tools were designed for compatibility with the *IEM Plugin Suite*, which served as a reference for the spatialization logic, while incorporating directivity control and modulation features derived from Joshua Parmenter's *SuperCollider* plugins. Developed within the *shoat* library, these objects were designed to integrate more seamlessly with *Pure Data* and *Plugdata* patches than standard VST hosting via [vstplugin~]⁹.

Similarly, the development of the *conTorchionist*¹⁰ library was intertwined with the compositional process. Notably, the refinement of the library's core Real Fast Fourier Transform (RFFT) and Inverse RFFT (IRFFT) processors—which embed windowing, overlap, and normalization logic—was largely conducted during experiments for the piece. In *second nature*, the [torch.rfft~] and [torch.irfft~] objects are used alongside custom processors to create a spectral analysis and resynthesis workflow that generates hybrid soundscapes where natural, anthropogenic, and synthetic materials coalesce. For example, the [torch.spectrals~] processor identifies prominent frequency peaks to create “frozen” spectral trails that linger after the original source has ceased. This processor emerged from the compositional desire to make bird calls within the soundscape persist, creating a sonic texture that also generates the harmonic content for the choir. Since the soundscape was recorded via *Ambisonic* microphone, the [torch.amb.spectrals~] processor was developed to spatialize these trails, allowing them to be frozen in both time and space (Figure 3).

9 The *shoat* library is available at <https://github.com/zepadovani/shoat>. Accessed on February 18, 2026.

10 The *conTorchionist* library is available at <https://github.com/ecrisufmg/contorchionist>. Accessed on February 3, 2026.

Figure 4 - The [torch.mic_array_spectral_analyzer~DSP logic



Source: Elaborated by the authors.

The processor analyzes the RFFT signals from a quadraphonic microphone array to detect the spatial and spectral footprint of the surrounding sound field.

As a primary conceptual pillar of the piece involves collective listening and interaction via the *rói-róis*, a specific analysis workflow was developed to track the spectral footprint of these instruments. This process is anchored by the [torch.mic_array_spectral_analyzer~] processor, a custom C++ object that performs a real-time breakdown of the sound field captured by a quadraphonic microphone array according to user-defined

frequency bands. The analyzer's output is coupled with the `ctgui.speccrader` object, which plots the location and intensity of detected sounds, and a *corpus matcher* system. This system manages a database of pre-analyzed animal sound samples, triggering the most spectrally similar sample and positioning it spatially according to the detected input. This interactive process enables the performers and the audience to trigger the bio-acoustic sounds of a curated corpus, creating a hybrid soundscape that blurs the boundaries between natural and artificial, while fostering a collective engagement with the sonic environment (Figure 4).

4.2 Neural spatialization and control: applying LSTM and C-VAE models in *Eutrópia III*

Eutrópia III (2025), for flute and live electronics, was composed in 2025 and is part of a cycle of five pieces that uses Eutropia (one of the cities described by Marco Polo to Emperor Kublai Khan in Italo Calvino's book *Invisible Cities*) as a poetic image. The conception of this cycle of pieces stems from a poetic interpretation created by the description of Eutropia. Each of the five pieces in the cycle, composed for different instrumental combinations, works as one of the possible Eutropias — one unfolding and transforming from the materials of the previous one through processes of rewriting; quotation; translation; expansion; unfolding; transcription; commentary; reflection, among others, carried out on the original material itself and also on the compositional process as a whole. This piece was composed within the context of the research and development of the *conTorchionist* library; thus, many of the machine learning related processors, algorithms, and *Pure Data* objects employed in *Eutrópia III* were designed specifically to address some creative and technical questions encountered during the compositional process.

A core functionality of *conTorchionist* lies in its capacity for real-time inference using pre-trained neural network models.

As detailed in Section 3.3.2, the [torch.ts] object facilitates the deployment of sophisticated architectures, such as *Variational Autoencoders* (VAEs) and *Transformers*, directly within computer music environments. This integration enables the design of complex interactive processes by embedding deep learning algorithms as foundational components of interactive musical systems. This paradigm is explored in *Eutrópia III* (2025), for flute and live electronics, which employs two distinct pre-trained models: a *Long Short-Term Memory* (LSTM) network (Hochreiter; Schmidhuber, 1997) for generating spatial trajectories¹¹, and a *Transformer-based Conditional Variational Autoencoder* for the real-time control of interactive processes¹². The development of these compositional strategies followed a unified prototyping and testing workflow across *Python* and *Pure Data*, leveraging the compatibility and interconnectivity between *PyTorch* and *conTorchionist* libraries. These models were designed, trained, and validated in *Python*, then exported via *TorchScript* for execution within the *Pure Data* environment.

The choice for the LSTM model, in particular, to create and manipulate sound trajectories addresses the inherent temporal complexity that spatialization processes present. This architecture was originally designed to solve the vanishing and exploding gradient problem in classical *Recurrent Neural Networks* (RNN) caused by long temporal sequences¹³. Since spatial trajectories are dynamic events that unfold over time, they require a robust long-term memory mechanism to be modeled properly. In this sense, the LSTM provides the technical means to preserve temporal information in long sequences due to its gated *memory cell* logic and its capacity to control which

¹¹ The *Python* code for the LSTM model is available at: https://github.com/oviniuscscar/LSTM_Spat_Trajectories

¹² The *Python* code for the *Transformer-based Conditional Variational Autoencoder* model is available at: https://github.com/oviniuscscar/Transformer_C-VAE

¹³ During the weight update process in traditional RNN models, the gradient between two instants t and k depends on the recurring weight w multiplied several times, resulting in w^{t-k} . If $|w| < 1$, this value tends to decrease rapidly (the gradient disappears). Conversely, if $|w| > 1$, it increases uncontrollably (the gradient explodes). Since this occurs when $t-k$ is large, traditional RNNs have problems “learning” long-term temporal dependencies (Raschka *et al.*, 2022, p. 525).

temporal information should be “remembered”¹⁴. Choosing this architecture is appropriate because spatial trajectories change continuously and internally. The movements in a trajectory depend on both the current state and a history of previous displacements, which provides the context needed for modeling.

However, the model implemented for *Eutrópia III* is slightly different from standard LSTM architectures and their conventional applications. To create a more dialogic interaction, an embedding layer manipulation mechanism was integrated into the model’s architecture. This modification enables not only the precise reproduction of learned trajectories but also the creation of intermediate behaviours and transitions absent from the original training data. Consequently, this design redefines the model’s primary function, shifting from a narrow focus on temporal prediction toward an interactive framework wherein the embedding layers serve as an exploratory field of compositional potential.

The model was built with 2 LSTM layers created with 256 hidden states¹⁵, an embedding layer, and 3 final linear layers with tanh activation at the output. Thus, the input data is sent to the LSTM layers, which process it sequentially and keep the internal state updated at each temporal step. At the end of each input sequence, the hidden state of the LSTM corresponding to the last step of the input sequence is concatenated with the output of the embedding layer, which receives a vector with the trajectory type in one-hot format. This creates an organized control space based on the three trajectory types present in the training data, allowing for their practical manipulation in real time contexts. Finally, the tensor resulting from the concatenation is processed by the linear layers, which return the model’s prediction.

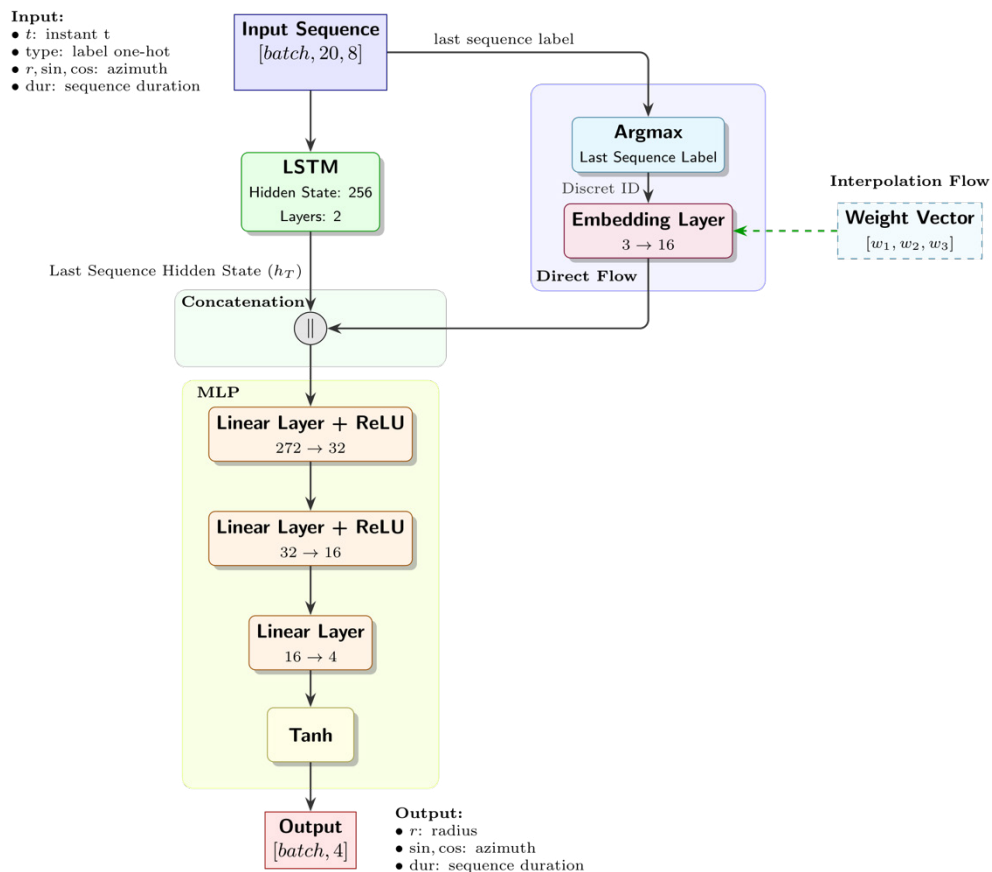
¹⁴ The LSTM architecture employs a mechanism composed of three gates that managements the information flow: the *forget gate* filters the previous *memory cells* information, the *input gate* modulates the integration of new information, and the *output gate* determines the information to be exported as the hidden state to the next *memory cell*

¹⁵ A LSTM layer with 256 hidden states consequently, has 256 *memory cells*

To train the LSTM model, a dataset containing three types of spatial trajectories (lissajous, spiral, and circular) was created. Each trajectory is represented as a sequence of temporal steps, where each step includes features such as time, a one-hot encoded vector indicating trajectory type (Lissajous: [1, 0, 0], spiral: [0, 1, 0], circular: [0, 0, 1]), radius, azimuth (expressed as separate sine and cosine values), and duration. The model was trained to predict the next step in the trajectory from the current input, allowing it to learn the specific patterns and dynamics of each trajectory type.

This architecture supports two distinct inference modalities. The primary method processes input data through the complete network layers to yield predicted values for radius, azimuth, and duration. Conversely, the second method facilitates enhanced parametric control by allowing direct manipulation of the embedding layer weights. This approach enables the generation of interpolated trajectories; for instance, applying a weight vector of [0.5, 0.5, 0] prompts the model to generate hybrid trajectories that bridge the characteristics of lissajous and spiral trajectories. Similarly, a vector of yields a trajectory between spiral and circular types. In specific sections of *Eutrópia III*, three descriptors — inharmonicity, spectral flux, and attack time — are extracted from the flute performance in real time to serve as the weight vector for the embedding layer. Consequently, the generated spatial trajectories dynamically modulate the flutist's gestural nuances, fostering an interaction between the instrumental writing and the interactive spatialization process. The diagram in figure 5 illustrates the model's architecture, featuring the two modes of inference.

Figure 5 - LSTM model architecture for spatial trajectory



Source: Elaborated by the authors.

The input sequence is processed by the LSTM, and the resulting hidden state is concatenated with a controllable embedding layer (discrete or interpolated). An MLP then predicts the next coordinate (radius, azimuth) and duration.

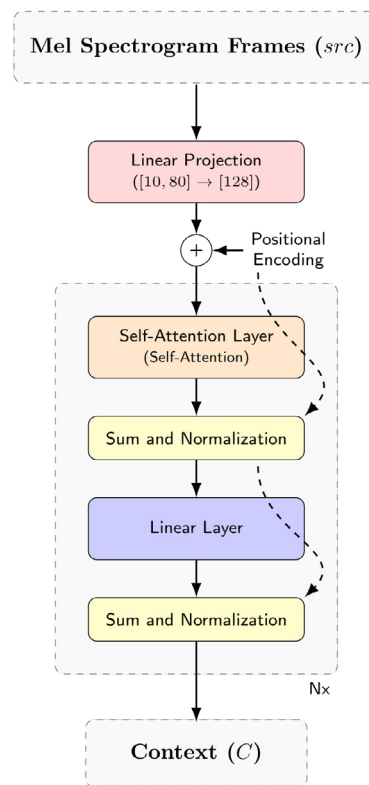
The *Transformer-based Conditional Variational Autoencoder* model was used in *Eutrópia III* to build an interactive mapping process to control parameters of a sampler module. The implemented *transformerC-VAE* process has a hybrid architecture designed for sequence-to-sequence (*seq2seq*) mapping tasks, which allows to correlate spectral characteristics of the input audio with control parameters for a sampler process. The model also uses a probabilistic latent space to capture the implicit stochastic variability present in sound gestures. Thus, the architecture is composed of three main interconnected modules: (1) *Conditional Encoder*; (2) *Variational Encoder*; (3) *Generative Decoder*.

In contrast to recurrent (e.g., RNNs, LSTMs) or convolutional (e.g., CNNs) architectures, the *Transformer* relies fundamentally on attention mechanisms, as introduced by Vaswani *et al.* (2017). Its primary distinction lies in the self-attention layers, which enable the model to capture global dependencies across an entire sequence simultaneously, thereby discerning context and relational meaning with greater efficiency than sequential processing. While traditionally applied to *Natural Language Processing* (NLP) tasks, the architecture implemented for *Eutrópia III* adapts this framework to process sequences of Mel spectrograms and control parameters. This adaptation leverages the *Transformer's* capacity to model complex temporal dependencies and long-range structural patterns within musical data. Furthermore, the integration of a variational component allows the model to map input data into a continuous, probabilistic latent space, thereby facilitating the precise manipulation, generation, and controlled variation of the output.

The processing flow of the *Transformer C-VAE* begins in the *Conditional Encoder*, responsible for extracting a robust latent representation (context C) from the input sequence, composed of 10 *frames* of Mel spectrogram, each containing 80 frequency *bins*. This module uses the architecture of the traditional *Transformer Encoder*, incorporating *sinusoidal positional coding*¹⁶ to preserve the temporal order of the audio frames. The *encoder* consists of a stack of layers that apply attention mechanisms followed by linear layers and residual connections, generating a sequence of context vectors C that will serve as memory for the following steps. Figure 6 shows a diagram of the *Conditional Encoder* architecture.

¹⁶ Since *transformer* models process the entire input sequence in parallel and not step-by-step as in RNN models, they do not have the temporal context explicitly. Sinusoidal positional coding is employed to solve this problem, injecting a pattern of sine and cosine waves into each spectrogram input to indicate its relative position. Thus, for each position in the input sequence and each dimension of the embedding vector, the added value is calculated for even dimensions: $(2i)$: $PE(pos, 2i) = \sin(pos/10000^{2i/dmodel})$ and for odd dimensions: $(2i+1)$: $PE(pos, 2i+1) = \cos(pos/10000^{2i/dmodel})$ (Vaswani *et al.*, 2017, p. 6)

Figure 6 - *Conditional Encoder* architecture



Source: Elaborated by the authors.

Conditional Encoder processing 10 Mel spectrogram frames. It uses sinusoidal positional encoding and self-attention layers with residual connections to generate context vectors (C) encapsulating spectral features.

The stochastic core of the model is based on a *Variational Encoder* (VAE) architecture, which uses the mapping of a continuous latent space, denoted by z , through the interaction between the *posterior* and *prior* distributions. During the training phase, the *posterior* distribution is fed by the concatenation between the target sequence *tgt* (control parameters) and the context C , the latter derived from the conditional encoder. This data stream is processed by attention layers and subjected to a temporal pooling mechanism, which consolidates the latent information along the chronological dimension. The module results in the parameterization of a multivariate Gaussian, $q(z|x,C)$, defined by the mean (μ) and log-variance ($\log \sigma^2$)

vectors. In parallel, the model integrates a *prior* network, whose function is to estimate the latent distribution using the input context C strictly. The relevance of the *prior* becomes central in the inference stage: given the absence of the target sequence at this stage, the subsequent module remains inactive. Thus, the *prior* ensures the predictive capacity of the system, allowing the sampling of latent vectors z that preserve semantic and structural consistency from the frames of the Mel spectrogram and the corresponding context C . Figure 7 illustrates the *Variational Encoder* module.

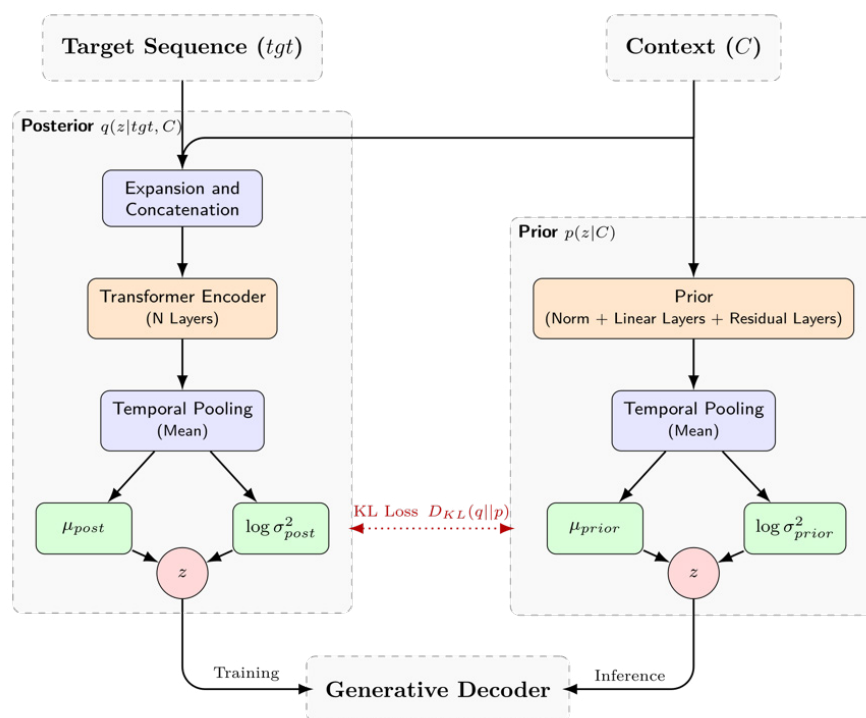


Figure 7 - Variational Encoder architecture

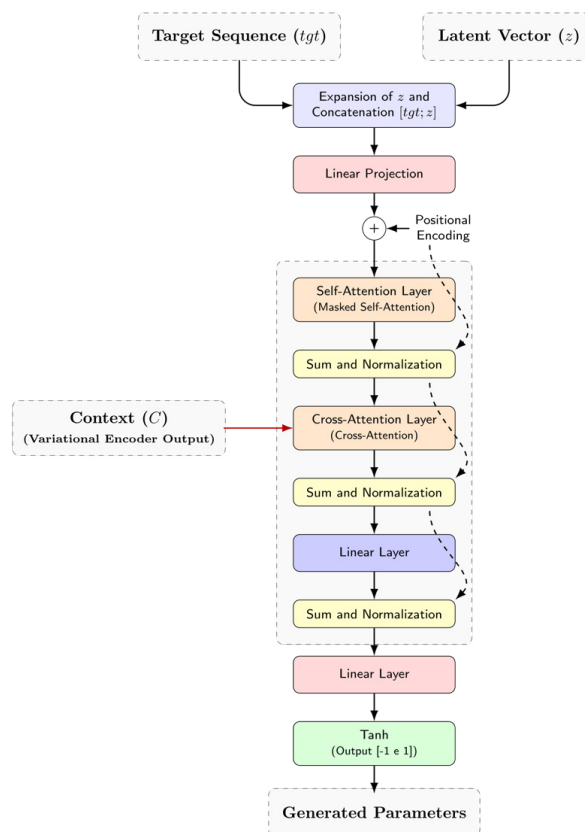
Source: Elaborated by the authors.

Variational Encoder mapping context (C) and target (tgt) to latent space (z). A posterior network parameterizes a Gaussian distribution ($\mu, \log \sigma^2$), while a prior network estimates the distribution from context alone for inference.

The subsequent reconstruction of the target parameters is performed by the *Generative Decoder*, a module that, unlike conventional *Transformer* architectures, has been adapted to integrate the latent vector z through temporal expansion

and subsequent concatenation to the input sequence. During training, the module employs a *Teacher Forcing*¹⁷ strategy to optimize convergence. The *Generative Decoder* architecture uses two distinct attention mechanisms: masked self-attention, which imposes a temporal causality constraint by preventing access to future steps in the sequence, and cross-attention, responsible for aligning the generation with the global context C . Finally, the output is projected to the dimensionality of the target parameters through a linear layer, followed by a tanh activation that restricts the values to the interval $[-1,1]$, enabling subsequent scaling to the scales of each parameter (Figure 8).

Figure 8 - *Generative Decoder* architecture



Source: Elaborated by the authors.

Generative Decoder reconstructing synthesis parameters. It uses masked self-attention for autoregression and cross-attention for context alignment (C), integrating the latent vector (z) to project the output sequence.

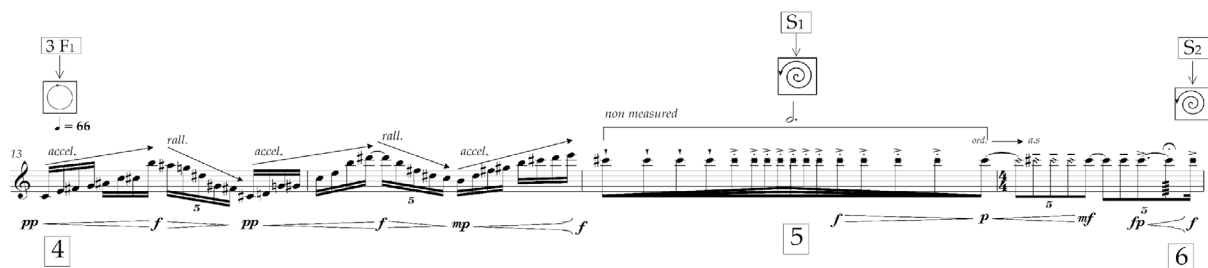
¹⁷ Teacher forcing involves using the ground truth data from time $t-1$ as input for prediction at time t , aiming to stabilize convergence and mitigate the accumulation of sequential errors, although it establishes an operational discrepancy in relation to inference, in which the model acts in a purely autoregressive manner.

The Mel spectrogram frames that compose the dataset were extracted from a flute sound corpus encompassing articulations such as *staccato*, *legato*, *frullato*, and multiphonic sounds. Each input sequence corresponds to a vector of 20 target parameters, organized in 3 dimensions: pitch (7 variables in MIDI cents), amplitude (7 variables in MIDI velocity), and texture (6 variables, encompassing four metronome intervals, grain size, and interval scope). This structure allows the model to correlate spectral characteristics with specific parametric signatures. For example, the *frullato* class is characterized by fast rhythmic bursts (metronome values between 236–474 ms), short grains, and intense dynamics. In contrast, multiphonics exhibit dilated metronome intervals (1000–3849 ms), generating more rarefied temporal intervals.

The model's output is mapped to control parameters of a sampler module based on the *Trevor Sampler*, as employed by Philippe Manoury in *Pluton* (1988) (Clarke; Dufeu; Manning, 2020). Our implementation modifies the original mechanism to feature two banks of eight sample players, enabling the simultaneous playback of sound fragments captured in real time during the performance itself. This sonic material is triggered by four independent metronomes, resulting in a four-level polyrhythmic texture. Beyond temporal triggering, other parameters control the overall playback duration, pitch, and velocity of the sample playback. The four metronomic values are derived from the model's outputs, while grain size, a set of pitch values, and amplitude dictate the duration, pitch, and intensity of the samples, respectively. This mapping strategy fosters a dynamic, gesture-responsive interaction, yielding a texture that evolves according to the spectromorphologic characteristics of the flute as processed by the model. Moreover, the probabilistic nature of the latent space z enables exploration of internal variability within gestural classes and supports continuous transition between different sonic states. In turn, this allows the generation of nuances and hybrid sound textures.

This operational mechanism is exemplified in the opening section of *Eutrópia III* (see Figure 9). At bar 15 (electronic event 5), the sampler module is activated, capturing a fragment of the flute's slap tongue gesture. Concurrently, the *Transformer C-VAE* processes the Mel spectrogram frames of the input signal to generate target parameters associated with the slap tongue class, which in turn modulate the sampler's behavior. The resulting output is a dense, rapid rhythmic "cloud" of iterative flute articulations. Simultaneously, the LSTM model generates a spiral spatial trajectory, utilizing the features extracted from the slap tongue gesture as the weight vector for the embedding layer that interpolates the spiral trajectory patterns. The same interaction process occurs at bar 16 (electronic event 6).

Figure 9 – *Eutrópia III*, excerpt



Source: Elaborated by the authors.

Excerpt of the first section of *Eutrópia III* (2025), for flute and live electronics. Numbers inside squares indicate electronic events.

5 Conclusion

In this article, we have argued for a fundamental shift in how we approach machine listening and learning technologies in creative and research contexts, advocating for a move away from consumerist and alienating relationships with technology toward one of critical appropriation and technodiversity. The *conTorchionist* library is, in this sense, more than just a flexible and nomadic tool for cross-platform and multi-architectural processes related machine listening/learning; it represents an infrastructure aimed at seizing the technical means of artistic creation and research, enabling artists to appropriate, adapt,

and reinvent their tools in accordance with their specific creative needs, insights, and contexts. As an open-source and flexible software, the library is designed to facilitate direct engagement with its underlying logic, enabling users to interpret and extend its functionality in accordance with their creative and investigative needs.

Through the frameworks of *research-development-creation* and *practice-based research*, exemplified by the works *second nature* and *Eutrópia III*, we demonstrated how the development of tools like *conTorchionist* and other technical artifacts can be an integral part of the creative process, rather than a mere technical ancillary. In the context in which it becomes possible to code with the assistance of agentic AI, we proposed the concept of *vibe composing* as a practice that integrates the development of technical tools into the compositional process itself. By treating the coding of algorithms as a fundamental creative gesture, *vibe composing* allows for a more intimate and inventive relationship with technology, where the act of coding becomes a poetic gesture that shapes the aesthetic outcome in profound ways.

This approach gains particular relevance when considering the material conditions of the library's production. Developed by a team of only two researcher-developers-artists within the South American academic context, *conTorchionist* illustrates how *vibe composing* can serve as a vital resource in scenarios marked by limited funding and human resources. Here, agentic AI functioned as a force multiplier, enabling a small team to bridge complex low-level programming language, structure a flexible and modular architecture, and create tools that are compatible with multiple real-time and non-real-time environments. This situated appropriation of AI resources into practical tool-building for artistic creation highlights how seizing the means of technological production can be a powerful strategy for artists and researchers, particularly in contexts where access to resources is constrained.

Undoubtedly, these new practices of appropriation and invention are not without challenges. For instance, the most effective models for coding with agentic AI are still produced by large technology companies, with limited transparency regarding the sources of their training data, model architectures, and inference logic. Furthermore, self-hosted models require expensive hardware and technical expertise, which can present a barrier for many artists. However, such engagement with AI tools appears less constrained than the use of closed *SaaS* platforms designed for artistic production; these often homogenize creative processes and aesthetic outcomes, alienating artists from the technical objects that mediate their work and fostering a passive consumer relationship with technology. The strategy of using AI to build tools for music production, rather than relying on closed AI services with very limited affordances, offers a pragmatic pathway for artists to face the challenges of the current technological landscape. By engaging directly with the code and the underlying logic of these systems, artists can resist the passive role of consumers within the black box economy, asserting their epistemic and technical sovereignty.

In future work, we aim to continue developing *conTorchionist* within research-development-creation and practice-based research scenarios, exploring the creative possibilities and technical innovations inherent in this approach. As most wrappers are currently in early development, with full implementation primarily available for *Pure Data*, we will prioritize the expansion of *Max*, *SuperCollider*, and *Python* interfaces. Furthermore, we are dedicated to producing comprehensive tutorials, documentation, and practical examples. These resources are designed to lower the barrier to entry, empowering artists to find their own methods of appropriating the machine listening and learning processes embedded in the library. By enhancing our distribution strategies, we seek to foster a more inclusive and accessible ecosystem for researchers and practitioners alike.

References

- BOGDANOV, D.; WACK, N.; GÓMEZ, E.; GULATI, S.; HERRERA, P.; MAYOR, O.; ROMA, G.; SALAMON, J.; ZAPATA, J.; SERRA, X. **ESSENTIA: An Open-source Library for Sound and Music Analysis**. *Em: PROCEEDINGS OF THE 21ST ACM INTERNATIONAL CONFERENCE ON MULTIMEDIA 2013*, New York, NY, USA. ISBN: 978-1-4503-2404-5. DOI: 10.1145/2502081.2502229. Disponível em: <http://doi.acm.org/10.1145/2502081.2502229>. acessado: 25 mar. 2018
- BREGMAN, A. S. **Auditory Scene Analysis: The Perceptual Organization of Sound**. Cambridge, Mass: MIT Press. 1990
- BRENT, W. Cepstral Analysis Tools for Percussive Timbre Identification. *Em: PROCEEDINGS OF THE 3RD INTERNATIONAL PURE DATA CONVENTION 2009*, São Paulo. Disponível em: <http://www.williambrent.conflations.com/papers/features.pdf>. acessado: 11 jun. 2014
- BROWN, G. J.; WANG, D. **Computational Auditory Scene Analysis: Principles, Algorithms, and Applications**. Piscataway: Wiley-IEEE Press. 2015
- CAESAR, R. A Escuta, Entre o Robô e o Animal. **Revista Portfolio EAV**. v. 1, 1. 2013
- CAILLON, A.; ESLING, P. **Streamable Neural Audio Synthesis With Non-Causal Convolutions**. 2022. DOI: 10.48550/arXiv.2204.07064. Disponível em: <http://arxiv.org/abs/2204.07064>. acessado: 29 jun. 2025
- CAILLON, A.; ESLING, P. **RAVE: A Variational Autoencoder for Fast and High-Quality Neural Audio Synthesis**. 2021. DOI: 10.48550/arXiv.2111.05011. Disponível em: <http://arxiv.org/abs/2111.05011>. acessado: 29 jun. 2025
- CAMPOLO, A.; CRAWFORD, K. Enchanted Determinism: Power without Responsibility in Artificial Intelligence. **Engaging Science, Technology, and Society**. v. 6, p. 1–19. 2020

CANDY, L.; EDMONDS, E.; VEAR, C. Practice-Based Research. **Em:** VEAR, C. (org.). Abingdon, Oxon ; New York, NY: Routledge, 2022. p. 27–41

CLARKE, M.; DUFEU, F.; MANNING, P. **Inside Computer Music**. New York: Oxford University Press. 2020

COLLINS, N. Machine Listening in SuperCollider. **Em: The SuperCollider book**. WILSON, S.; COLLINS, N.; COTTLE, D. Cambridge, Mass: MIT Press, 2011. p. 439–461

COLLINS, N. SCMIR: A SuperCollider Music Information Retrieval Library. **International Computer Music Conference Proceedings**. v. 2011. 2011

COLLINS, N. Musical robots and listening machines. **Em: The Cambridge Companion to Electronic Music**. COLLINS, N.; ESCRIVAN, J. (orgs.). Cambridge: Cambridge University Press, 2007. 1ª Edição. 171–184

COLLINS, N. **Towards Autonomous Agents for Live Computer Music: Realtime Machine Listening and Interactive Music Systems**. (tese de doutorado). Cambridge: Faculty of Music, University of Cambridge. 2006

DOWNIE, J. S. Music Information Retrieval. **Annual review of information science and technology**. v. 37, 1, p. 295–340. 2003

EMMERSON, S. Listening With Machines: A Shared Approach. **Organised Sound**. v. 20, 1, p. 68–75. 2015

EMMERSON, S. **Living Electronic Music**. Burlington: Ashgate. 2007

FRANK, M.; ZOTTER, F.; SONTACCHI, A. Producing 3D Audio in Ambisonics. **Em: AUDIO ENGINEERING SOCIETY CONFERENCE: 57TH INTERNATIONAL CONFERENCE: THE FUTURE OF AUDIO ENTERTAINMENT TECHNOLOGY–CINEMA, TELEVISION AND THE INTERNET 2015**,. Disponível em: https://iaem.at/Members/zotter/2015_ambisonics_aes_hollywood.pdf/@download/file/2015_Ambisonics_AES_Hollywood.pdf. acessado: 8 dez. 2025

GOSSELIN, P.; LE COGUEC, É. **La Recherche Création: Pour Une Compréhension de La Recherche En Pratique Artistique**. Québec: PUQ. 2006

GRÄF, A.; SCHOEN, T.; WESCH, B. Pd-Lua Signals and Graphics. *Em*: PROCEEDINGS OF THE 19TH LINUX AUDIO CONFERENCE 2025, Lyon, France. Disponível em: <https://hal.science/hal-05096052>. acessado: 17 fev. 2026

HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**. v. 9, 8, p. 1735–1780. US. 1997

HUI, Y. **Machine and Sovereignty: For a Planetary Thinking**. Minneapolis: University of Minnesota Press. 2024

HUI, Y. **Art and Cosmotechnics**. Minneapolis, MN Minneapolis, MN: e-flux Distributed by the University of Minnesota Press. 2021

HUI, Y. **Tecnodiversidade**. São Paulo: Ubu Editora. 2020

HUI, Y. **Recursivity and Contingency**. London Lanham: Rowman & Littlefield International, Ltd. 2019

HWANG, J.; HIRA, M.; CHEN, C.; ZHANG, X.; NI, Z.; SUN, G.; MA, P.; HUANG, R.; PRATAP, V.; ZHANG, Y. TorchAudio 2.1: Advancing Speech Recognition, Self-Supervised Learning, and Audio Processing Components for PyTorch. *Em*: 2023 IEEE AUTOMATIC SPEECH RECOGNITION AND UNDERSTANDING WORKSHOP (ASRU) 2023,. Disponível em: <https://ieeexplore.ieee.org/abstract/document/10389648/>. acessado: 28 jun. 2025

JEHAN, T. **Creating Music by Listening**. Massachusetts Massachusetts Institute of Technology. 2005

LACHENMANN, H. Über Das Komponieren. Wiesbaden: Breitkopf & Härtel, 1996. p. 73–82

LEWIS, G. E. Too Many Notes: Computers, Complexity and Culture in “Voyager”. **Leonardo Music Journal**. v. 10, p. 33–39. 2000

LEWIS, G. E. Interacting with Latter-Day Musical Automata.

Contemporary Music Review. v. 18, 3, p. 99–112. 1999

MALT, M.; JOURDAN, E. Real-Time Uses of Low Level Sound Descriptors as Event Detection Functions Using the Max/MSP Zsa.Descriptors Library. p. 12. 2009

MARX, K. **Grundrisse Foundations of the critique of political economy ; (Rough draft).** Harmondsworth, Baltimore, London: Penguin Books New Left Review Penguin Books Australia. 1973

MCFEE, B.; RAFFEL, C.; LIANG, D.; ELLIS, D. P.; MCVICAR, M.; BATTENBERG, E.; NIETO, O. Librosa: Audio and Music Signal Analysis in Python. *Em: PROCEEDINGS OF THE 14TH PYTHON IN SCIENCE CONFERENCE 2015,*

MOORE, T.; BRADBURY, J.; TREMBLAY, P. A.; GREEN, O. Making Machine Learning Musical: Reflections on a Year of Teaching FluCoMa. **Journal SEAMUS.** v. 32. 2021.

PADOVANI, J.; DE OLIVEIRA, V. conTorchionist: A flexible nomadic library for exploring machine listening/learning in multiple platforms, languages, and time-contexts. *Em: PROCEEDINGS OF THE 19TH BRAZILIAN SYMPOSIUM ON COMPUTER MUSIC.* Campinas: SBCM, 14 set, 2025

PASZKE, A.; GROSS, S.; MASSA, F.; LERER, A.; BRADBURY, J.; CHANAN, G.; KILLEEN, T.; LIN, Z.; GIMELSHEIN, N.; ANTIGA, L.; DESMAISON, A.; KÖPF, A.; YANG, E.; DEVITO, Z.; RAISON, M.; TEJANI, A.; CHILAMKURTHY, S.; STEINER, B.; FANG, L.; BAI, J.; CHINTALA, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Red Hook, NY, USA: Curran Associates Inc., 2019. p. 8026–8037

PUCKETTE, M. **Sigmund~.c.** Código fonte. 2005. Disponível em: <https://github.com/pure-data/pure-data/blob/master/extra/sigmund%7E/sigmund%7E.c>. acessado: 16 fev. 2026

PUCKETTE, M. Pure Data: Recent Progress. *Em*: PROCEEDINGS OF THE THIRD INTERCOLLEGE COMPUTER MUSIC FESTIVAL 1997. Disponível em: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=fc33440116d6731554335fba4bcee0f74d99f43e>. acessado: 28 jun. 2025

RASCHKA, S.; LIU, Y.; MIRJALILI, V.; DZHULGAKOV, D. **Machine Learning with PyTorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python**. Birmingham: Packt Publishing. 2022

RAY, P. P. A Review on Vibe Coding: Fundamentals, State-of-the-art, Challenges and Future Directions. 2025

REGILME, S. S. F. Artificial Intelligence Colonialism: Environmental Damage, Labor Exploitation, and Human Rights Crises in the Global South. **SAIS Review of International Affairs**. v. 44, 2, p. 75–92. 2024

ROWE, R. Split Levels: Symbolic to Sub-symbolic Interactive Music Systems. **Contemporary Music Review**. v. 28, 1, p. 31–42. 2009

ROWE, R. **Interactive Music Systems: Machine Listening and Composing**. Cambridge: MIT press. 1992

SIMONDON, G. **Du mode d'existence des objets techniques**. Paris: Aubier. 1989

SRNICEK, N. **Platform Capitalism**. Cambridge, UK ; Malden, MA: Polity. 2017

STIEGLER, B. **For a New Critique of Political Economy**. Tradutor: Daniel Ross. Cambridge: Polity. 2010

SUPERCOLLIDER. **SuperCollider Documentation**. 2025. Disponível em: <https://docs.supercollider.online/Browse.html#UGens%3EAnalysis>. acessado: 28 jun. 2025

TREMBLAY, P. A.; GREEN, O.; ROMA, G.; BRADBURY, J.; MOORE, T.; HART, J.; HARKER, A. Fluid Corpus Manipulation Toolbox. 2022. Disponível em: <https://doi.org/10.5281/zenodo.6834643>. Acessado: 2 fev. 2026

TREMBLAY, P. A.; GREEN, O.; ROMA, G.; HARKER, A. From Collections to Corpora: Exploring Sounds through Fluid Decomposition. *Em: INTERNATIONAL COMPUTER MUSIC CONFERENCE AND NEW YORK CITY ELECTROACOUSTIC MUSIC FESTIVAL 2019*,. Disponível em: <https://pure.hud.ac.uk/en/publications/from-collections-to-corpora-exploring-sounds-through-fluid-decomp>. acessado: 11 out. 2023

TREMBLAY, P. A.; ROMA, G.; GREEN, O. Enabling Programmatic Data Mining as Musicking: The Fluid Corpus Manipulation Toolkit. **Computer Music Journal**. v. 45, 2, p. 9–23. 2021

TURKLE, S. **The Second Self: Computers and the Human Spirit**. 20th anniversary ed., 1st MIT Press ed. Cambridge, Massachusetts: MIT Press. 2005

VAROUFAKIS, Y. **Technofeudalism: What Killed Capitalism**. London: The Bodley Head. 2023

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention Is All You Need. **Advances in neural information processing systems**. v. 30. 2017

WANG, D.; BROWN, G. J. (ORGS.). **Computational Auditory Scene Analysis: Principles, Algorithms, and Applications**. Hoboken, N.J: Wiley-IEEE Press. 2006

YANG, Y.-Y.; HIRA, M.; NI, Z.; ASTAFUROV, A.; CHEN, C.; PUHRSCHE, C.; POLLACK, D.; GENZEL, D.; GREENBERG, D.; YANG, E. Z. TorchAudio: Building Blocks for Audio and Speech Processing. *Em: ICASSP 2022-2022 IEEE INTERNATIONAL CONFERENCE ON ACOUSTICS, SPEECH AND SIGNAL PROCESSING (ICASSP) 2022*,. Disponível em: <https://ieeexplore.ieee.org/abstract/document/9747236/>. acessado: 28 jun. 2025

Acknowledgments

The authors would like to thank FAPEMIG, CAPES and CNPq for their financial support. Also, we would like to thank the

developers of the open-source libraries and tools mentioned in this paper, as they were not only a source of inspiration but also a reference for the design of conTorchionist. We deeply thank Miriam Akkermann, Michael Käßler, the singers of the Singakademie Dresden, Mia Hadzikadunic, Ugur Can Akkaya, and Tristan O'Leary for their invaluable collaboration and support during the composition of *Second Nature* and its performances in Dresden and Berlin.

As an aid for synthesizing appropriate English phrasing and for text revision, the writing of this work utilized the Google Gemini 3.0 Pro and qwen2.5:14b models, which functioned strictly as supportive writing tools.

Publisher

Federal University of Goiás. School of Music and Performing Arts. Graduate Program in Music. Publication in the Portal of Periodicals UFG.

The ideas expressed in this article are the responsibility of their authors and do not necessarily represent the opinion of the editors or the university.